```
# Practical 10: Ensemble Learning

# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    ConfusionMatrixDisplay
)

# 2. Load the dataset
data = load_breast_cancer(as_frame=True)

X = data.data
y = data.target

print("Dataset: Breast Cancer Wisconsin Diagnostic")
print("Number of records:", X.shape[0])
```

```
print("Number of features:", X.shape[1])

print("\nClass Distribution:")
print(y.value_counts().rename(index={
    0: "Malignant",
    1: "Benign"
}))

# 3. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# 4. Create machine learning models
models = {
    "Logistic Regression (Baseline)": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=5000, random_state=42)
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
```

```
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100,
        random_state=42
    )
}

# 5. Train models and evaluate performance
results = []
trained_models = {}

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    trained_models[name] = model

    results.append({
        "Model": name,
        "Accuracy (%)": accuracy_score(y_test, y_pred) * 100,
        "Precision (%)": precision_score(y_test, y_pred) * 100,
        "Recall (%)": recall_score(y_test, y_pred) * 100,
        "F1-Score (%)": f1_score(y_test, y_pred) * 100
    })

# 6. Display performance comparison
results_df = pd.DataFrame(results).set_index("Model")

print("\n--- MODEL PERFORMANCE COMPARISON ---")
print(results_df.round(2))
```

```
# 7. Plot performance comparison graph
results_df.plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Baseline vs Ensemble Learning Techniques")
plt.ylabel("Performance Score (%)")
plt.xlabel("Machine Learning Models")
plt.ylim(0, 105)
plt.xticks(rotation=15, ha="right")
plt.legend(loc="lower right")
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

# 8. Display confusion matrices
for name, model in trained_models.items():

    print("\nConfusion Matrix:", name)

    ConfusionMatrixDisplay.from_estimator(
        model,
        X_test,
        y_test,
        display_labels=data.target_names
    )

    plt.title(name)
    plt.tight_layout()
```

```
    plt.show()

# 9. Display classification reports
for name, model in trained_models.items():

    print("\nClassification Report:", name)

    print(classification_report(
        y_test,
        model.predict(X_test),
        target_names=data.target_names
    ))

# 10. Identify the best model using F1-score
best_model = results_df["F1-Score (%)"].idxmax()

print("\n--- BEST MODEL ---")
print("Best Model:", best_model)

print(
    "F1-Score:",
    round(results_df.loc[best_model, "F1-Score (%)"], 2),
    "%"
)
```